The Staff Engineer's Path: You're a Role Model Now (Sorry)

An excerpt from The Staff Engineer's Path by Tanya Reilly. The two paths, looking ahead, and creating future leaders.

GERGELY OROSZ
DEC 14, 2022

👋 *Hi, this is Gergely with a bonus, free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at big tech and high-growth startups through the lens of engineering managers and senior engineers.*

If you're not a subscriber, here are recent issues you missed:

- *Inside the performance calibration process.*
- *Performance calibrations at tech companies: part one.*
- *The 2022 hiring market, as seen by tech recruiters*
- *The Scoop #34: uncertainty and anxiety at Amazon and Google, budget cuts at Spotify.*

Subscribe to get weekly issues. Many subscribers expense this newsletter to their learning and development budget. 👇

A book that I have been waiting for a long time is finally out: The Staff Engineer's Path, written by Tanya Reilly, Senior Principal Engineer at Squarespace.

My copy of The Staff Engineer's Path.

In the article Engineering career paths at Big Tech and high-growth startups, I covered the most typical career paths in tech companies, including the Staff Engineer one. The Staff engineer role is one that is less explored, and less documented. Before Tanya's book, the only other in-depth resource I could recommend was the book Staff Engineer by Will Larson. However, Tanya's book is the first one which provides a type of manual of how to thrive at the Staff level.

I was lucky enough to get my hands on a copy before it was published, and read the book before it came out. Here's my review of the book, which is also inside the print edition:

> "If you're a senior engineer wondering what the next level is, a staff-level engineer or a manager of staff engineers, this book is for you. It covers so many of the things no one tells you about this role-things that take long years, even with great mentors, to discover on your own."

The book is now out, and I asked Tanya if she'd be open to sharing a chapter from the book, which she agreed to. I chose two sections: Two Paths, and Look Ahead. In this issue, we'll cover:

1. **Two paths.** An introduction on why the Staff path is less clear, even when you are in it.
2. **You're a Role Model Now (Sorry).** The blessing and the curse of the staff role: rather than guiding you, people will look to you for guidance.
3. **Look Ahead.** Approaches on how to plan for a longer time horizon: a skill that becomes essential at the staff level.

My usual disclaimer: as with all my recommendations, I was not paid to recommend this book, and none of the links are affiliate ones. See my ethics statement for more details.

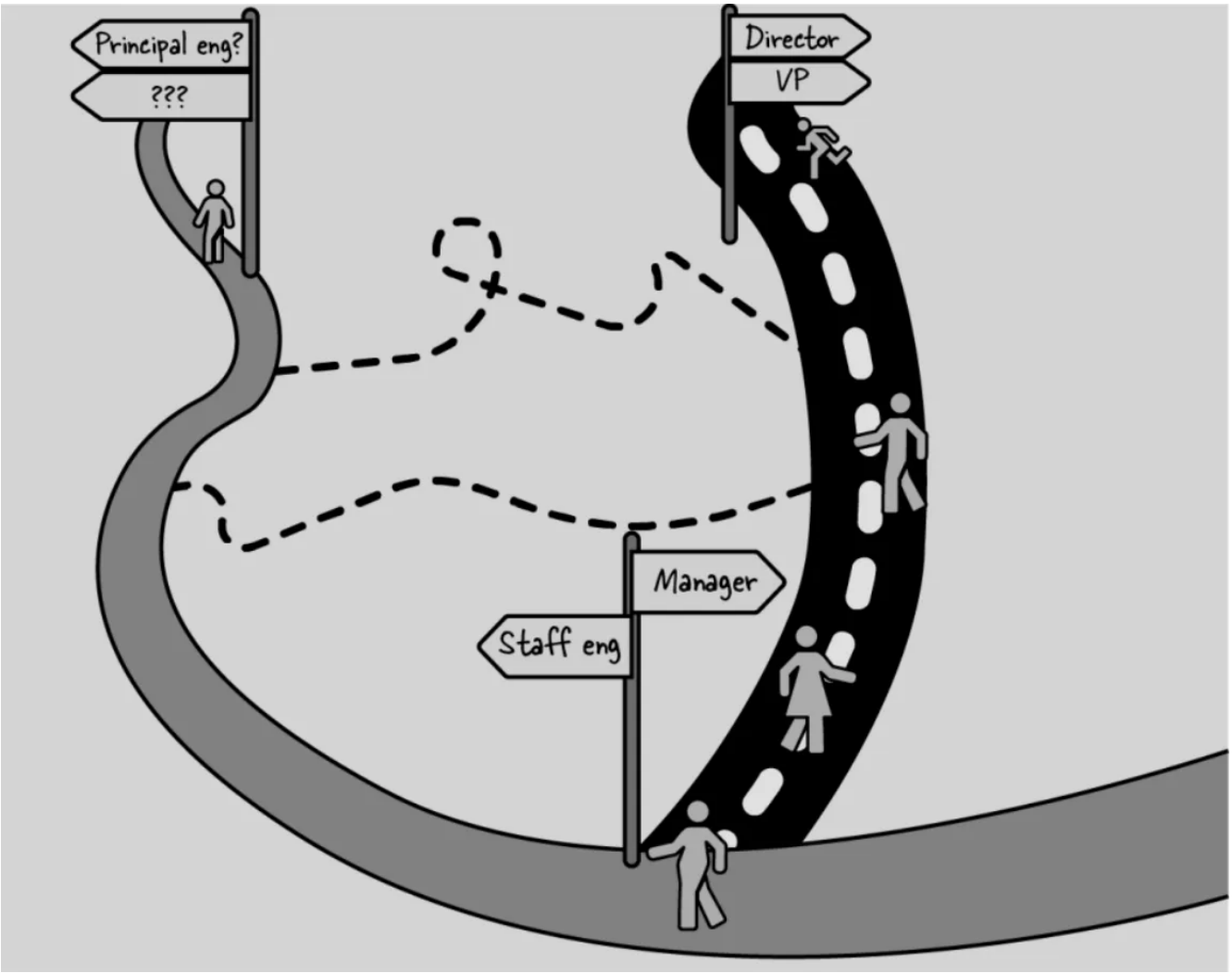
The below excerpts are from The Staff Engineer's Path, by Tanya Reilly. Copyright © 2022 Tanya Reilly. Published by O'Reilly Media, Inc. Used with permission.

1. Two Paths

This section is how [The Staff Engineer's Path] book starts.

You may find yourself at a fork in the road (Figure P-1), two distinct paths stretching ahead. On one, you take on direct reports and become a manager. On the other, you become a technical leader without reports, a role often called staff engineer.

If you really could see five years ahead on both of these paths, you'd find that they have a lot in common: they lead to many of the same places, and the further you travel, the more they'll need many of the same skills. But at the start they look quite different.



A fork in the road. Source: The Staff Engineer's Path

The manager's path is clear and well traveled. Becoming a manager is a common, and perhaps default, career step for anyone who can communicate clearly, stay calm during a crisis, and help their colleagues do better work.

Most likely, you know people who have chosen this path. You've probably had managers before, and perhaps you have opinions about what they did right or wrong. Management is a well-studied discipline, too. The words *promotion* and *leadership* are often assumed to mean "becoming someone's boss," and airport bookshops are full of advice on how to do the job well. So, if you set off down the management path, it won't be an easy road, but you'll at least have some idea of what your journey will be like.

The staff engineer's path is a little less defined. While many companies now allow engineers to keep growing in seniority without taking on reports, this "technical track" is still muddy and poorly signposted.

Engineers considering this path may have never worked with a staff engineer before, or might have seen such a narrow set of personalities in the role that it seems like unattainable wizardry. (It's not. It's all learnable.) The expectations of the job vary across companies, and, even within a company, the criteria for hiring or promoting staff engineers can be vague and not always actionable.

Often the job doesn't become clearer once you're in it. Over the last few years, I've spoken with staff engineers across many companies who weren't quite sure what was expected of them, as well as engineering managers who didn't know how to work with their staff engineer reports and peers. All of this ambiguity can be a source of stress. If your job's not defined, how can you know whether you're doing it well? Or doing it at all?

Even when expectations are clear, the road to achieving them might not be. As a new staff engineer, you might have heard that you're expected to be a technical leader, make good business decisions, and influence without authority—but how? Where do you start?

Note from Gergely: the book itself attempts to answer this last question.

2. You're a Role Model Now (Sorry)

This is an excerpt from the beginning of Chapter 7: You're a Role Model Now (Sorry) in the book.

"Don't think out loud," my friend Carla Geisser warned me when I became a staff engineer. "You'll find out a month later that people are talking about your half-baked idea like it's already a project." My colleague Ross Donaldson described his own role even more starkly: "Being staff doesn't absolve you of being wrong, but it does mean you need to be careful when you open your dang mouth."

This is the blessing and the curse of a staff engineer title: people will assume you know what you're talking about—so you'd better know what you're talking about! Your work will be a little less checked and your ideas considered more credible. Rather than guiding you, people will look to you for guidance.

What Does It Mean to Do a Good Job?

Most of all, you'll be a role model. How you behave is how others will behave. You'll be the voice of reason, the "adult in the room." There will be times when you'll think "This is a problem and someone should say something"...and realize with a sinking feeling that that someone is you. When you model the correct behavior, you're showing your less experienced colleagues how to be a good engineer. Later, in Chapter 8, we'll look at how to actively, deliberately influence your organization and colleagues for the better. But this chapter is about passive influence, the kind that you have just by the way you act as an engineer and as a person.

Values are what you do

Your company might have a written definition of what good engineering means: written values, perhaps, or engineering principles. But the clearest indicator of what the company values is what gets people promoted. No matter how much your organization claims to encourage collaboration and teamwork, that message will be undermined if any staff engineers get to that level through "heroic" solitary efforts. If your engineering principles describe a culture of thorough code reviews, but senior engineers approve PRs without reading them, everyone else will rubber-stamp code reviews too. The work

that you do is implicitly the type and standard of work that others will see as correct and emulate.

Engineering goes beyond what you do when you're talking to computer systems; it's also about how you talk to humans. So sometimes being a good engineer boils down to being a good colleague. If you're mature, constructive, and accountable, you're telling your new grads that's what a senior engineer does. If you're condescending, impossible to please, or never available, that's what a senior engineer does, too. You shape your company every day, just by how you behave.

But I don't want to be a role model!

Being a role model is not always comfortable. But as you become more senior, it's one of the biggest ways you'll have an impact. Like it or not, you're setting your engineering culture. Take that power seriously. Being a role model doesn't mean you have to become a public figure, be louder than you're comfortable with, or throw your weight around. Many of the best leaders are quiet and thoughtful, influencing through good decisions and effective collaboration (and showing fellow quiet people that there's space for them to lead).

If the idea of being a leader is terrifying, you may need to build up to it. Start small. Maybe compliment someone's success on a public channel, or offer to help onboard a new person. Think of leadership as a skill to build, just like you would learn a new language or technology. The more you practice, the easier it will be.

Be the best engineer and the best colleague that you can be. Do a good job and let others see it. (And help others do the same! We'll discuss how in Chapter 8.) That's what being a role model is.

3. Look Ahead

This is an excerpt from the middle of Chapter 7: You're a Role Model Now (Sorry) in The Staff Engineer's Path.

While there are, as you've seen, times when your first priority will be to get something to market quickly, most of the time you're planning for a longer time horizon. The code

and architecture you work on are likely to still be in use in 5 or 10 years. The interconnected software systems that make up your production environment may last much longer, and each component will influence the ones that follow. (Think of it as a Ship of Theseus: every individual component may get replaced over the years, but the fundamental system continues.) As Titus Winters writes in Software Engineering at Google (O'Reilly) "Software engineering is programming integrated over time." Expect the impact of your software to stick around.

Your organization, codebase, and production environment probably existed before you joined them. They'll probably exist after you move on. Don't optimize for now at the cost of future velocity or engineering ability. It's OK to plant some seeds that you won't personally see grow.

Here are a few ways you should be thinking beyond the current moment.

Anticipate what you'll wish you'd done

Remember our question from in Chapter 3: "What will Future You wish Present You had done?" When you're making plans or doing work, consider your future self and your future team to be stakeholders: after all, they'll have to deal with whatever decisions you make now.

Telegraph what's coming

Be clear about what your broad direction is, even if you don't know the details yet. Here's an example: teams sometimes avoid announcing deprecation dates for old systems, because they're not quite ready to begin the major migration to the new system. But you can announce the intention to deprecate it. If everyone knows a migration will begin in a year or two, new projects will know not to invest in it. Some teams may find themselves with free time and move to the new system without you even asking them. A small amount of work now will set people's expectations, save their time, and make your future deprecation project a little easier.

Tidy up

Have you ever had to work in a tool shed where the last person didn't clean up after themselves? It's horrible. You grab the drill and the battery's out of power. The safety goggles aren't in their case; you search through three boxes before finding them with the sander. The floor is covered in detritus. There is no flow state in an environment like that. Everything takes three times as long as it should.

Now think about what it's like when every tool you want is at arm's reach. Your workflow just works. So take the time to leave your production environment, codebase, or documentation so that it *just works* for whoever comes along next. Write tests that will let you refactor your code without breaking things. Follow your style guide so that the people who copy your approach will also be following your style guide. Leave no traps, like dangerous scripts that everyone needs to remember not to run or configurations that are changed locally but not updated in source control. Make it safe to move around.

Keep your tools sharp

Don't just tidy up: continually invest in making your environment better. If you can move quickly and safely, you'll spend less time on repetitive work and you'll be able to do more. Increasing your velocity increases your reliability, too: every minute you shave off your time to detect a problem or deploy a fix is a minute you've taken off every outage.

Look for optimizations that will let you build, deploy, and release more quickly: smaller builds, intuitive tooling, fixing or deleting flaky tests, repeatable processes, automation everywhere. Be judicious about where you invest: building tooling, platforms, or processes take time, so choose the optimizations that will genuinely make a difference.

Create institutional memory

Every time someone leaves your company, you lose institutional knowledge. If you're lucky, you have some old-timers storing history in their brains. But eventually, inevitably, you'll have complete staff turnover. When an old system breaks, there'll be nobody left to say "Oh, yes, I remember when we ran into this before. Here's what we did last time."

My ex-colleague John Reese, at the time a principal engineer at Google, often also took the role of systems historian: he curated a record of how the site reliability organization had evolved and how running software in production had changed over the years. To create institutional memory, he wrote in-depth articles about the parts of the ecosystem he knew best, then interviewed others to uncover the past, documenting formative systems and practices. Although he's moved on from Google now, that history lives on with a new set of curators.

While most organizations don't have someone deliberately writing down their history (though maybe we should!), you can send information into the future by writing things down. This includes decision records that explain what you were thinking, systems diagrams that include the obvious things that "everyone knows," and code comments that include context on what's going on. However you create the history, include searchable keywords so that future people have some chance of understanding what you did and why—and think about what you know that future people might not.

Expect Failure

My all-time favorite incident retrospective is the one Fran Garcia wrote about his then-employer, Hosted Graphite, being taken down by an AWS outage. The reason I love this one is that Hosted Graphite didn't use AWS, so the team was quite surprised at being affected by its outage. They had no way of predicting it.

How many unpredictable failures like that lurk in your systems? Assume it's a lot. The network will fail, the hardware will fail, the people will have an off day. There will always be bugs. Odd interactions between parts of the system you haven't even thought about will cause problems.

You can't predict everything that will go wrong, but you can predict that *something* will go wrong. Plan for what you'll do when it does. Build the expectation of failure into your products: test the error paths as thoroughly as the success paths, and make the product do something sensible and user-friendly when it doesn't get the kind of response it expects. Make sure you'll find out when your systems aren't behaving, and have a plan for how you'll respond to it.

Plan in advance for major incidents by adding some conventions around how you work together during an emergency: introduce the incident command system I mentioned earlier, for example, and practice the response before you need it. Your disaster plans will invariably have something go wrong, so simulate disaster with chaos engineering tooling or controlled outages. Drills, game days, or tabletop exercises can let you uncover which parts of your response won't work. And of course, if you haven't tested restoring your backups, assume you don't have any backups.

Optimize for maintenance, not creation

Software is created once, but it will need to be maintained for years. If you've got a binary running in production, it will need monitoring, logging, business continuity, scaling, and so on. Even if you intend to never touch the code again, the technical or regulatory ecosystem may force you to care: think of all the old systems that needed to be updated for Y2K, to support IPv6 or HTTPS, or for compliance concerns like SOX, GDPR, or HIPAA. Those won't be our last disruptive changes. (2038 is coming!)

Software gets maintained for much longer than it takes to create it, so don't build code that's hard to maintain. Here are some ways you can help Future You and your future team.

Make it understandable

At the moment you create new code or design a new system, you understand it well. Probably the people on your team also have a strong mental model of how it works. Expect that knowledge to decay a little every day. The system will never again be as well understood as it is on the day it's created. If it's hard to understand then, good luck in two years, when something breaks and you're trying to load that mental model back into your brain.

You have two choices to let future people understand your system.

One option is to focus on education and hands-on experience. You can run continual classes about the system, making sure that everyone who might have to work on it in future is fully trained and has logged enough hours to handle any problems that might arise.

The other option is to make it as easy as possible for people to understand the system when they need it. That means writing documentation with that future person as the main audience: a clear, short introduction; at least one big, simple picture (use arrows to show which direction data moves); links to anything they might wonder about. Then expose the system's inner workings as clearly as possible. Make it possible to see what it's doing, through tooling, tracing, or useful status messages. Make your systems observable: easy to inspect, analyze, and debug. And keep them simple, which I'll talk about next.

Keep it simple

There's a Martin Fowler quote that I love: "Any fool can write code that a computer can understand. Good programmers write code that humans can understand." (*Refactoring: Improving the Design of Existing Code by Martin Fowler et al (Addison-Wesley))* Senior engineers sometimes think they can demonstrate their prowess with the flashiest, most complicated solutions. But it's easier to make something complicated. It's much harder to make it simple!

How can you make something simple? Spend more time on it. When you think of a solution to the problem you're working on, treat it as "just the first." Spend at least the same amount of time on another solution. Now that you understand it better, see if you can make it simpler: fewer lines of code, fewer branches, fewer teams, fewer hours of maintenance, fewer running binaries, fewer files touched. The longer the system is intended to last, the longer you should spend trying to make it as simple as you can. Make it easy to build mental models of the system or the code.

Beware of organizations that seem to reward complexity. Ryan Harter, a staff data scientist, has written about how he's seen people create complicated solutions to prove that they're doing hard work. "I've seen folks slip machine learning into places it doesn't belong to get a flashy launch." He cautions, "Really, what we should want are simple solutions to complex problems. The complexity of our work is a cost to bear, not something to maximize!"

When you're dealing with inherently complex problems, make a deliberate decision about where in the system you're going to put the complexity: that one terrifying

module with the inscrutable business logic or performance optimizations. Make it so that someone looking at the entire system can treat that component as a magic black box and reason about everything else, so that there's a single place to go to when it's time to understand and modify the complex part.

Build to decommission

Someday your system will be turned off. How hard is that going to be for the people working on it then? Will they have to dig deep into the logic of other systems, unwinding tendrils that touch business logic and tracing into other systems to understand what data they're accessing? Or will there be a clean interface and a simple cutover?

Your architecture will evolve, and your components will settle into the middle. While it might be faster now for you to just wire in the new system, library, or framework, think about what will happen afterward. Will it be possible to replace it later without demolishing whatever other people have built on top of it?

Imagine knowing that you *personally* will need to decommission this component in 10 years. Future You won't be any less busy than Present You, so what can you do to help them out? Might you add a clean interface, make it easy to see which clients are still using a server, or design in a way that keeps a little distance between two systems that are being integrated? If you set out from the start to build a component that's easy to decommission, you'll have the side effect of building something modular and easy to maintain.

Create future leaders

Building up your team is an important part of future planning. It often will be easier and faster for you to solve problems or lead projects than for others to do it, but that doesn't mean you should take over. Your junior engineers are future senior engineers. Give them the space to learn, and opportunities to do hands-on work and solve increasingly difficult problems.

Chapter 8 will have a lot more about how to continually raise their skill levels.

I'll leave you with one more quote from John Allspaw's "On Being a Senior Engineer":

> "The degree to which other people want to work with you is a direct indication of how successful you'll be in your career as an engineer. Be the engineer that everyone wants to work with."

If you take nothing else away from this chapter, take that last sentence: the metric for success is whether other people want to work with you. If they don't, reevaluate your approach.

Takeaways

This is Gergely again.

Thanks again to Tanya for sharing a section of her book. The book is full of distilled mental models that I found nodding to, and practical guidance that I've yet to read elsewhere.

You can also follow Tanya on Mastodon, on Twitter or on LinkedIn. You'll also find her active on the Rands Leadership Slack community at times.

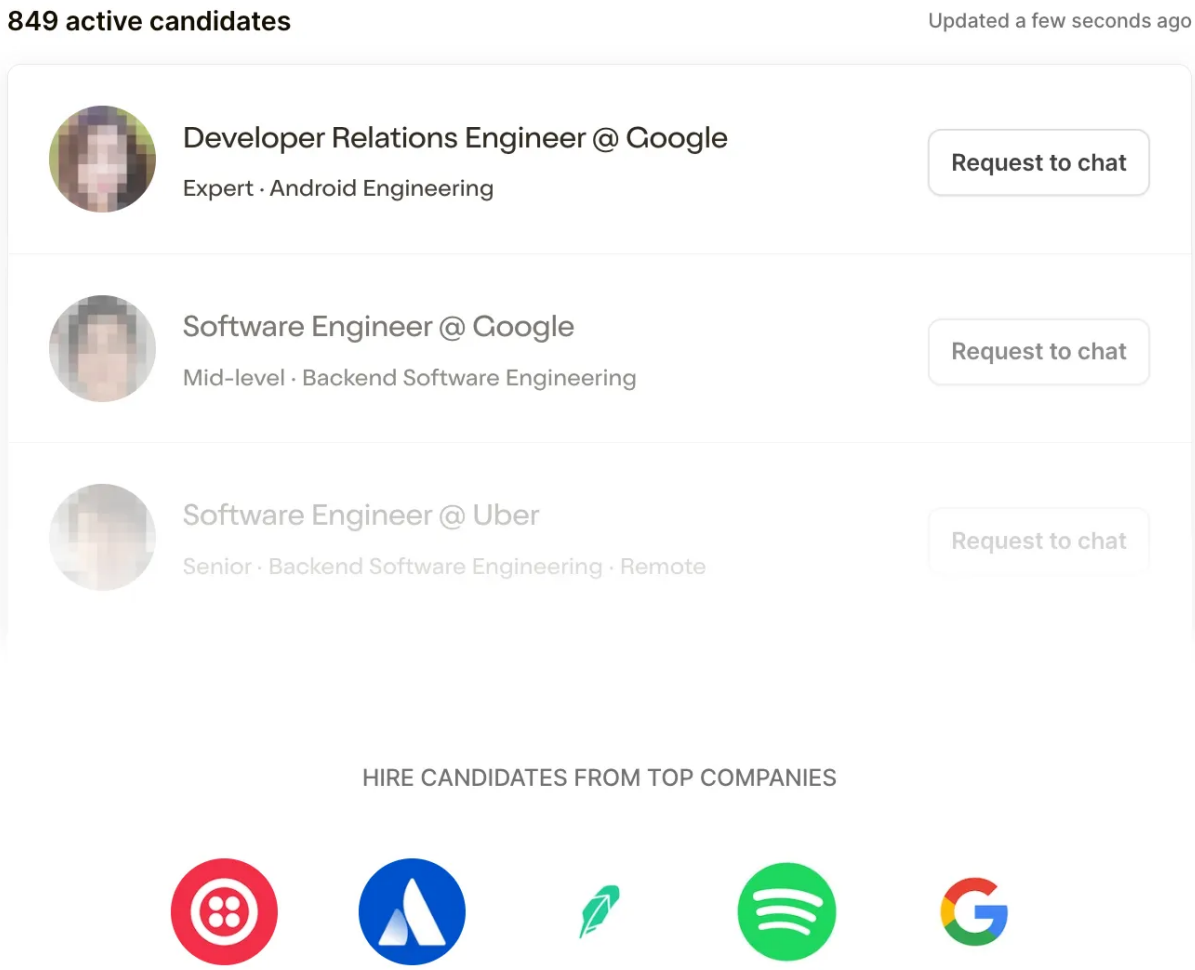
Looking ahead and trying to "predict the future" is what great staff engineers need to do. I really like the advice from Tanya on how you can get better at this:

- **Anticipate what you'd wish you'd done.** Consider your future self and your future team. How would your current decisions affect you? I love how, just following this train of thought, it's clear that you should tidy up after your self, and keep your tools sharp: meaning things like your builds fast, tests stable, deploys reliable.
- **Expect failure.** Less experienced software engineers are surprised when things actually fail: more experienced ones are not taken off-guard. Staff+ engineers need to get good at predicting what will fail and why it can happen.
- **Optimize for maintenance, not creation.** This thought deeply resonated with me. I've observed most software engineers take a thrill in building: and they

usually build quickly! However, it's maintenance where lots of pain follows. Great staff engineers build software in a way that will make it easy to maintain.

Join The Pragmatic Engineer Talent Collective

If you're hiring, join The Pragmatic Engineer Talent Collective to start getting regular drops of outstanding software engineers and engineering leaders who are open to new opportunities. It's a great place to hire developers - from backend, through fullstack to mobile - and engineering leaders.



If you're open for a new opportunity, join to get reachouts from vetted companies. You can join anonymously and leave anytime

170 Likes

6 Comments

Write a comment...

John S Dec 15, 2022

Hey Gergely, do you have any recommended resources for what it means to be a senior software engineer?

LIKE (8) REPLY

1 reply by Gergely Orosz

Bram Jan 5 ❤ Liked by Gergely Orosz

The URL for Fran Garcia's retrospective moved to https://blog.hostedgraphite.com/blog/spooky-action-at-a-distance-how-an-aws-outage-ate-our-load-balancer .

LIKE (1) REPLY

1 reply by Gergely Orosz

4 more comments...